

Using the MATLAB Editor

A Line-by-Line Walkthrough of the COE158 Activity Sheet

Contents

About This Worksheet	1
1. <code>clc, clear, close all</code>	1
2. <code>disp('Welcome to MATLAB')</code>	2
3. Saving as <code>first_script.m</code>	2
4. Variables and Arithmetic	3
5. Comments	3
6. Editing and Rerunning	4
7. Saving as <code>basic_calculations.m</code>	4
8. Understanding Errors	4
9. Sections with <code>%%</code>	4
10. Keyboard Shortcuts	5
11. Building Output Strings — <code>disp(['Sum = ', num2str(sum)])</code>	5
Summary — Quick Reference	6
Assignments and Practice Examples	6

About This Worksheet

This worksheet (COE158 — *Using the MATLAB Editor*) walks through your very first proper MATLAB script, piece by piece. It isn't testing anything complicated mathematically — the point of it is to get you comfortable with the **mechanics** of writing, saving, running, editing, and debugging a script in the Editor, since almost everything else you do in MATLAB this semester will build on these habits. Below, each command and idea from the worksheet is explained in the order it appears, followed by why it matters, and then a set of assignments at the end for extra practice.

1. `clc, clear, close all`

```
clc
clear
close all
```

These three lines almost always appear at the **very top** of a script, and the worksheet has you type them first for a reason: they reset your environment before your program

does anything else.

- **clc** — clears everything currently displayed in the Command Window. It does **not** delete any variables; it's purely cosmetic, so you're not scrolling past leftover output from an earlier run.
- **clear** — deletes all variables currently stored in memory (the Workspace). This matters because MATLAB keeps old variables around between runs unless you tell it not to — without **clear**, a bug in your new script could accidentally “work” only because a variable from a previous, unrelated run was still sitting in memory.
- **close all** — closes every open Figure window (any graphs you plotted earlier). Useful once your scripts start producing plots, so each run starts with a clean slate instead of stacking old graphs on your screen.

Why it's used here: even though this particular script doesn't plot anything yet, the worksheet is building the habit early — professional MATLAB scripts almost always start this way so that running the script twice in a row gives you the exact same result both times.

2. `disp('Welcome to MATLAB')`

```
disp('Welcome to MATLAB')
```

`disp` displays whatever you pass it directly, with no extra formatting or labelling. The text 'Welcome to MATLAB' is written in single quotes because it's a **char array** (see the earlier syntax notes) — MATLAB needs quotes around text so it knows you mean the literal words, not a variable named `Welcome`.

Why it's used here: it's the simplest possible way to prove your script actually ran. This is the MATLAB equivalent of a “Hello, World!” program — before worrying about calculations, you confirm the basic pipeline (write → save → run) works.

3. Saving as `first_script.m`

A MATLAB script must be saved with the **.m extension** before you can run it — this is what tells MATLAB “this file contains MATLAB commands.” The filename you choose (`first_script.m`) becomes how you'd refer to it if you ever wanted to call it from elsewhere, so filenames should be short, meaningful, and contain no spaces.

Why it's used here: this is the key difference the worksheet is highlighting between the **Editor** and the **Command Window** — code typed directly into the Command Window disappears once you close MATLAB, but a saved **.m** file can be reopened, rerun, and shared at any time.

4. Variables and Arithmetic

```
a = 20;  
b = 5;  
sum = a + b  
difference = a - b  
product = a * b  
quotient = a / b
```

a and b are variables storing the numbers 20 and 5. Notice the **semicolons** on the first two lines but **not** on the last four — this is deliberate, not a mistake. `a = 20;` and `b = 5;` just store values, and you don't need to see that confirmed on screen. But `sum = a + b` (no semicolon) is left open on purpose so that when you run the script, MATLAB immediately prints the result of each calculation to the Command Window — that's the entire point of this section, to see the arithmetic happen.

`+`, `-`, `*`, and `/` are the standard arithmetic operators — addition, subtraction, multiplication, and division. Because a and b here are single numbers (scalars) rather than matrices, these behave exactly like a calculator; the matrix-vs-element-wise distinction (`*` vs `.*`) only becomes relevant once you're working with vectors or matrices.

Note: using `sum` as a variable name here is fine for a short script, but be aware `sum` is also the name of a **built-in MATLAB function** (that adds up the elements of a vector). Naming your own variable `sum` temporarily “hides” that function within your script — usually harmless in a script this short, but a habit worth being careful with as your programs grow.

5. Comments

```
% Student Name  
% Index Number  
% COE152 MATLAB Practical  
% Addition of two numbers  
a = 20;  
b = 5;  
c = a + b;
```

Anything after a `%` symbol is a **comment** — MATLAB completely ignores it when running the script. Comments don't affect what the program does; they exist purely for humans reading the code (including *you*, revisiting your own script weeks later).

Why it's used here: the worksheet has you label the script with your name and index number as a comment specifically because this identifies your work without interfering with the program's output — unlike `disp('Student Name')`, a comment never appears in the Command Window when the script runs.

6. Editing and Rerunning

Changing `a = 20;` to `a = 100;` and rerunning the script demonstrates one of the main advantages of the Editor over the Command Window: you can modify **one line** and rerun the **entire script** instantly, rather than retyping every command from scratch. This is why scripts are so much more practical than working purely in the Command Window once your program grows beyond a couple of lines.

7. Saving as `basic_calculations.m`

Saving the edited script under a new, more descriptive filename (`basic_calculations.m`) reflects good practice: filenames should describe what the script actually does, not just be a generic placeholder like `first_script.m`. Reopening it afterwards to confirm it saved correctly is a habit worth keeping — it's much better to discover a saving problem immediately than the night before an assignment is due.

8. Understanding Errors

```
a = 10           % no semicolon -> value is displayed
a = ;           % syntax error -> nothing follows the '='
```

Removing the semicolon from `a = 10` isn't an error at all — it simply means MATLAB will display `a = 10` in the Command Window, which is expected behaviour, not a bug.

`a = ;`, on the other hand, **is** a genuine error: the assignment operator `=` expects a value on its right-hand side, and there isn't one. When you run this, MATLAB stops and shows a **red error message** in the Command Window, along with:

- The **line number** where the problem occurred (so you don't have to hunt through your whole script)
- A short **explanation** of what MATLAB expected instead

Why it's used here: deliberately breaking your own code is one of the fastest ways to learn to read MATLAB's error messages without panicking. Errors are a completely normal part of programming — the goal of this exercise is to get comfortable interpreting what MATLAB is telling you, rather than avoiding errors altogether (which is impossible!).

9. Sections with `%%`

```
%% Part A
a = 10;

%% Part B
```

```
b = 20;  
c = a + b
```

A double percent sign %% marks the start of a new **section** within a script. Visually, the Editor highlights each section with a light-yellow (or grey) background band so you can see the divisions at a glance. Clicking inside a section and choosing **Run Section** (or pressing Ctrl+Enter) runs **only that section**, using whatever variables already exist in the Workspace — it does not rerun the whole file from the top.

Why it's used here: sections let you test or rerun one part of a long script without waiting for the entire thing to execute again. This becomes genuinely useful once a script has an early section that loads or generates data (slow) followed by a later section you're still tweaking (fast) — you don't want to redo the slow part every single time.

10. Keyboard Shortcuts

Shortcut	Function
Ctrl + N	Create a new script
Ctrl + O	Open an existing script
Ctrl + S	Save the current script
Ctrl + R	Comment selected lines
Ctrl + T	Uncomment selected lines
F5	Run the current script
Ctrl + Z	Undo the previous action
Ctrl + Y	Redo the previous action

Most of these mirror shortcuts you already know from other programs (Ctrl+S to save, Ctrl+Z to undo). The two worth calling out specifically are **Ctrl+R** and **Ctrl+T**: instead of manually typing a % in front of every line you want to temporarily disable, you can **select several lines at once** and press Ctrl+R to comment all of them out in one go (and Ctrl+T to bring them back). This is extremely useful while debugging — you can “switch off” a block of code you suspect is causing a problem without deleting it.

11. Building Output Strings — disp(['Sum = ', num2str(sum)])

```
disp(['Sum = ', num2str(sum)])
```

This line looks more complicated than the earlier disp calls, so it's worth unpacking piece by piece:

- **num2str(sum)** converts the *number* stored in sum into *text*, since you can't directly join a number onto a string with square brackets otherwise.

- `['Sum = ', num2str(sum)]` uses square brackets to **concatenate** (join together) two pieces of text: the literal label 'Sum = ' and the converted number. The comma separates the two pieces being joined.
- `disp(...)` then displays the single, combined piece of text.

Why it's used here: `disp(sum)` alone would just print the number with no label (e.g. 25), which is fine for a one-off calculation but unhelpful once your script produces several results — you'd have no way of telling which number was which. Building a labelled string like "Sum = 25" makes your output self-explanatory.

Summary — Quick Reference

Concept	Syntax	Purpose
Reset environment	<code>clc, clear, close all</code>	Clear screen, variables, and figures before running
Display text/value	<code>disp(x)</code>	Print something to the Command Window, no formatting
Save a script	File → Save, .m extension	Store commands for reuse, unlike the Command Window
Suppress output	<code>;</code> at end of line	Hide a line's result; omit it to show the result
Arithmetic	<code>+</code> <code>-</code> <code>*</code> <code>/</code>	Basic calculations on numbers
Comment	<code>%</code>	Note ignored by MATLAB, for human readers only
Edit & rerun	Change code, press F5	Update a script without retyping everything
Syntax error	e.g. <code>a = ;</code>	MATLAB reports the line number and likely cause
Sections	<code>%%</code> , Run Section / Ctrl+Enter	Run part of a script independently
Comment block	Ctrl+R / Ctrl+T	Comment/uncomment several selected lines at once
Text + number	<code>['label ', num2str(x)]</code>	Join text and a converted number into one string

Assignments and Practice Examples

Try these on your own in the Editor. Save each as its own .m file with a sensible name, and remember to start each script with `clc; clear; close all;`.

Assignment 1 — Fix the Bug

The script below contains **three** deliberate mistakes (a missing semicolon that shouldn't matter, a genuine syntax error, and a comment written incorrectly). Type it in exactly as shown, run it, read the error message MATLAB gives you, and correct all the problems until it runs cleanly.

```
clc
clear
close all
// This script adds two numbers
x = 12
y = ;
z = x + y;
disp(['Result = ', num2str(z)])
```

Assignment 2 — Personal Calculator

Write a script called `my_calculations.m` that:

1. Starts with `clc; clear; close all;`
2. Stores your age and the current year as two variables (semicolons included, so nothing prints yet)
3. Calculates the year you were born
4. Displays the result using a labelled string, e.g. "Year of birth = 2006", using the `['...', num2str(...)]` pattern from Section 11

Assignment 3 — Sectioned Script

Write a single script with **three** `%%` sections:

- Section 1: store three numbers of your choice as variables `p`, `q`, and `r`
- Section 2: calculate their sum and their average (hint: $\text{average} = \text{sum} \div 3$)
- Section 3: display both results as labelled strings

Run each section independently using `Ctrl+Enter`, in order, and confirm the Workspace panel updates correctly after each one.

Assignment 4 — Comment Practice

Take any script you've already written and select every line except the `disp` statements. Use **Ctrl+R** to comment them all out at once, run the script, and observe what happens. Then use **Ctrl+T** to uncomment them again. Write one sentence explaining, in your own words, why this shortcut is faster than commenting lines out one at a time.

Assignment 5 — Reflection

In a few sentences each, answer:

1. Why does `disp('Student Name')` print differently from a comment like `% Student Name`, even though both mention the same text?
2. If you removed `clear` from the top of a script, describe a scenario where that could cause your script to give a misleading result.
3. Why is `a = ;` an error, but `a = 10` with no semicolon is not?